Merge Sort

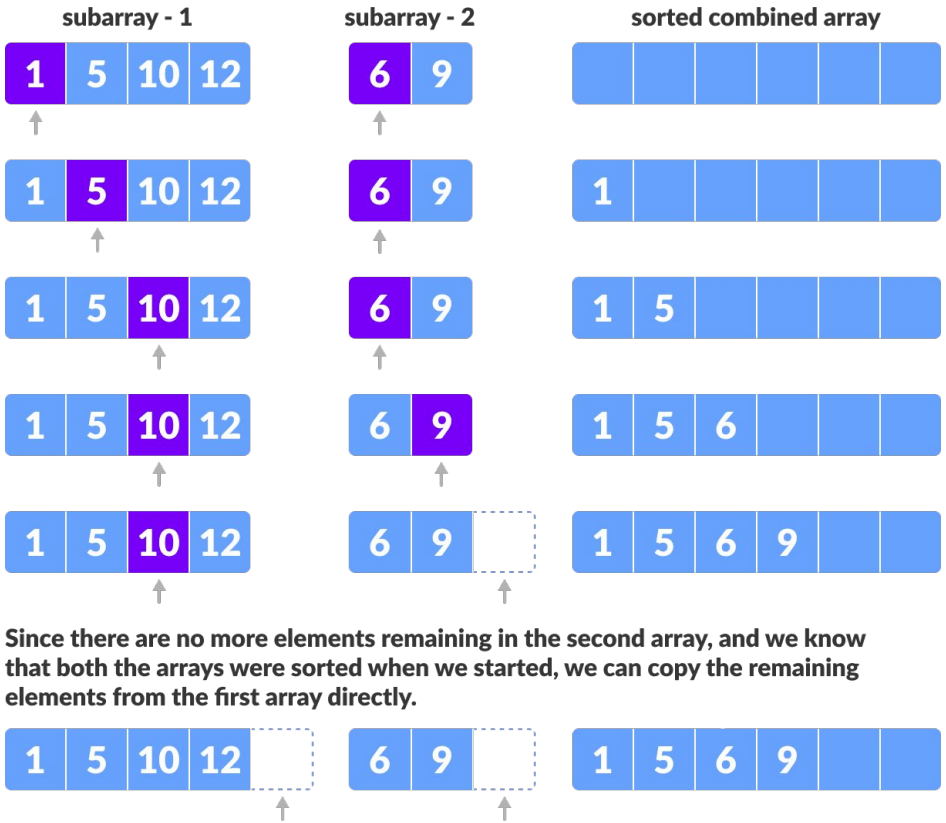
Do now

If we have 2 sorted arrays, how would you combine them to have only 1 sorted array?

Input: [4, 8, 9, 20, 25] [2, 5, 7, 12, 30]

Output: [2, 4, 5, 7, 8, 9, 12, 20, 25, 30]

Merging arrays



How can we write a merge method in Java?
Parameters?
Return?

How can we write a merge method?

```
public static int[] merge(int [] left, int[] right){ }
```

Or

```
public static void merge(int[] destination, int [] left,
                         int[] right){ }
```

Save here:

../APCSA/apcsa-assignments-spring-YourUsername/classwork/03_09_merge/Merge.java

Merge sort process

Recursive algorithm: It splits the array in half until it cannot be further divided.

Base case: When the array becomes empty or has only one element.

Recursive case: If the array has multiple elements, split the array into halves and recursively invoke the merge sort on each of the halves.

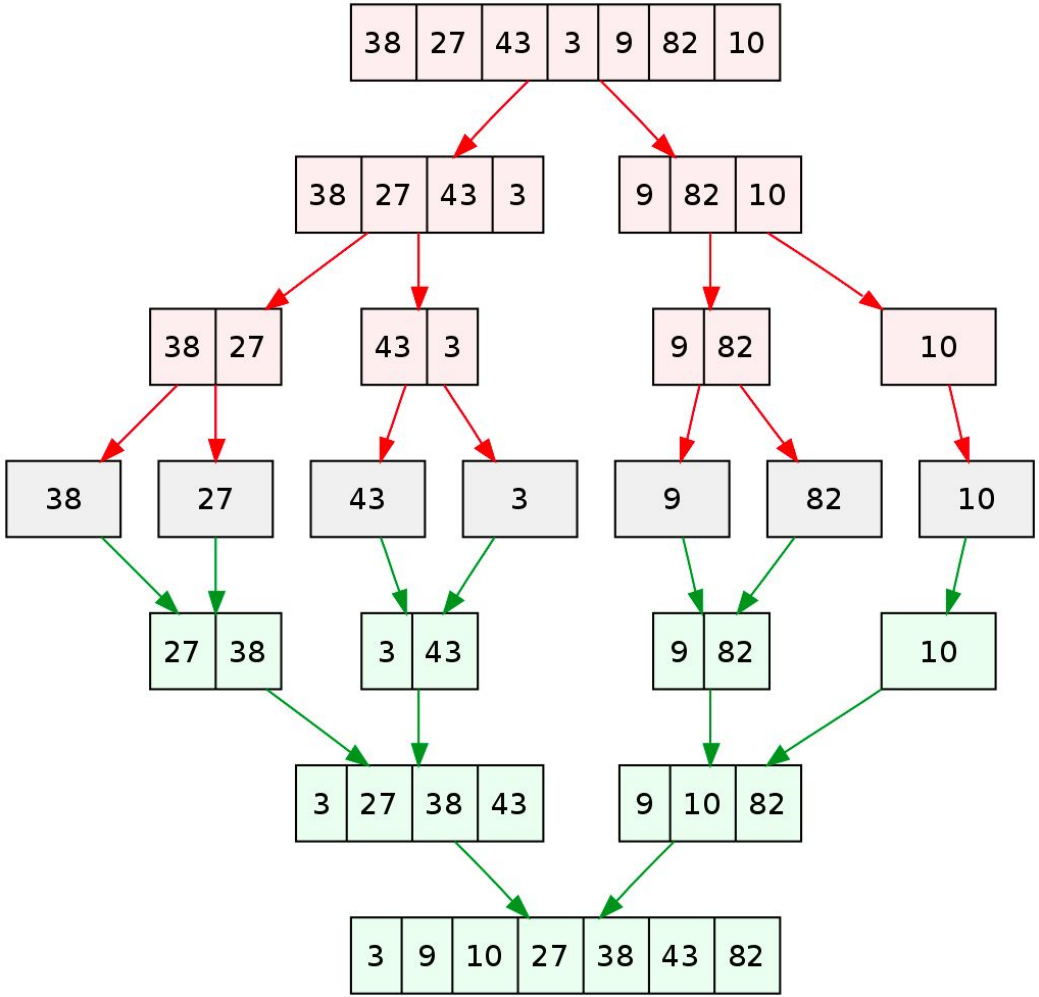
When both halves are sorted, the **merge operation** is applied, which means we have to **combine two smaller sorted array to eventually make a larger one**.

Merge Sort

Steps:

1. It divides and array into smaller subarrays.
2. It sorts each subarray.
3. It merges the sorted subarrays back together to form the final sorted array.

Merge Sort

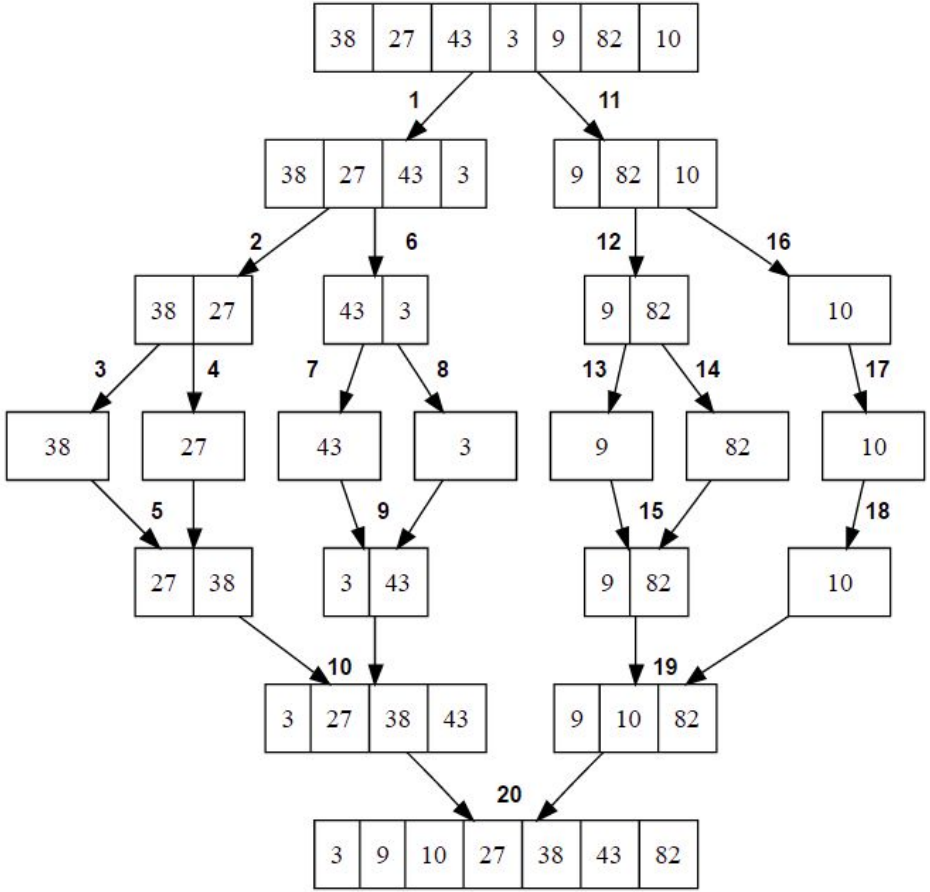


This algorithm is efficient because it quickly sort large arrays.

The red section is where the recursive calls split into sub-arrays. The green section would be when the values return and merge.

Time Complexity:

The list of size n is divided into a max of logn parts, and the merging of all sublists into a single list takes O(n) time. The worst, average and best cases have a run time of **O(n logn).**



Top-down view of the recursive merge sort algorithm.

Let’s implement the Merge Sort in Java

Lab is posted on our website: https://novillo-cs.github.io/apcsa/labs/

Method signature:

```
public static int[] mergeSort(int[] data){

}
```

Merge Sort Pseudocode

```
int[] mergeSort(data){
    if more than 1 element{
        L = mergeSort left side
        R = mergeSort right side
         return merge(L,R)
    }else{
        return data
    }
}
```

Let’s implement the Merge Sort in Java

Lab is posted on our website: https://novillo-cs.github.io/apcsa/labs/

Method signature:

```
public static int[] mergeSort(int[] data){

}
```

This is a **2-day project (3/3, 3/4)**

Today's homework: Dedicate 30 min to work on this lab at home.

Tomorrow: We will review any questions you might have and continue working on the code.

Optimized Merge Sort

Quick Review: How Classic Merge Sort Work

Merge sort is a divide-and-conquer algorithm. It repeatedly splits an array in half, recursively sorts each half, then merges the two sorted halves together.

Pseudocode:

```
mergeSort(array):
  if array.length <= 1 then return
  split array into LEFT and RIGHT halves
  mergeSort(LEFT)
  mergeSort(RIGHT)
  merge(LEFT, RIGHT) back into array
```

Time complexity: O(n log n)

Space complexity: O(n)

Why the classic merge sort version is slow?

The Problem with Classic Merge Sort

Problem 1: Excessive Memory Allocation

Every merge call allocates a new temporary array. New arrays are allocated and thrown away on every single merge.

Problem 2 — Extra cost on tiny subarrays

Merge sort recurses all the way down to single-element subarrays. But sorting 2, 3, or 4 elements with merge sort is enormous overkill — the recursive call overhead cost is significant..

Problem 3 — Merging Already-Sorted Halves

Suppose after the recursive calls you have LEFT = [1, 2, 3] and RIGHT = [4, 5, 6]. They are already in perfect order. Classic merge sort copies them both into a temp array and walks through every element anyway — completely wasted work.

How can we improve Merge sort?

With merge sort a subarray of 16 elements takes 4 recursive levels, but insertion sort would handle it in ~3 comparisons.

Use insertion sort for small subarrays. We can improve most recursive algorithms by handling small cases differently. Switching to insertion sort for small subarrays will improve the running time of a typical merge sort implementation by 10 to 15 percent.

How can we improve Merge sort if halves are sorted?

Skip the Merge When Already Sorted

After both halves are sorted, check: is arr[mid] <= arr[mid+1] (inplace algorithm)? If yes, the two halves are already in the correct relative order. You can skip the entire merge step for this call. Depending how you are implementing your code you may use the left and right arrays for the comparison left[last] <= right[first], then concatenate the arrays and return.

Adding this comparison before calling merge(). This single line can turn a nearly-sorted array from O(n log n) into effectively O(n).

Coding Time

Write an algorithm to improve the classic Merge Sort.

Time the execution of this new algorithm and compare the results with the classic solution (time java Driver)

Save here:

../APCSA/apcsa-assignments-spring-YourUsername/classwork/03_12_merge/FasterMergeSort.java

Complete the sorting algorithms summary.